CORRIGÉ · PROBLÈME · ANNALES SPÉ

Flots de données paresseux

Énoncé

OCaml exclusivement. `let list_finite = [1; 2; 3]` et `let rec list_ones = 1 :: list_ones`.

1. Croquis en maillons de `list_finite` et `list_ones` en mémoire ; quelle région de la mémoire ?

Le code suivant provoque `Stack overflow during evaluation (looping recursion?)` :

```
let list_fibo : int list =
  let rec list_fibo_with_internal_state (a:int) (b:int) : int list =
    a :: (list_fibo_with_internal_state b (a+b))
  in
  list_fibo_with_internal_state 0 1
```

1. Traduire *stack* ; expliquer l'erreur.
2. Avec `ocamlopt fibo.ml -o fibo` puis `./fibo` : l'une des étapes échoue-t-elle ?

`type 'a stream = Nil | Cons of 'a * (unit -> 'a stream)`. Exemples : `let rec ones = Cons (1, fun () -> ones)` ; `fibo` défini par `fibo_with_internal_state a b = Cons (a, fun () -> fibo_with_internal_state b (a+b))`.

1. Constante `integers : int stream` (entiers naturels croissants).
2. `range (a:int) (b:int) : int stream` (entiers de a inclus à b exclu).
3. Toute suite (σ_n) de chaînes est-elle codée par un `string stream` ?
4. `hd (u:'a stream) : 'a`, avec `Failure "hd"` si `u` est vide.
5. `tl (u:'a stream) : 'a stream`, avec `Failure "tl"`.
6. `of_list (l:'a list) : 'a stream`.
7. `iter (f:'a -> unit) (t:int) (u:'a stream) : unit` : applique `f` aux `t` premières valeurs existantes.
8. `map (f:'a -> 'b) (u:'a stream) : 'b stream`.
9. `zip (w:'a stream array) : 'a array stream` : flot des tableaux $[u_n, v_n, \dots]$, vide dès que l'un des flots l'est.

10. `intertwin (u:'a stream) (v:'a stream) : 'a stream` : alterne les éléments de `u` et `v`, en continuant avec l'autre quand l'un est épuisé.

```
let rec product_wrong u1 u2 =
  match u1, u2 with
  | _, Nil | Nil, _ -> Nil
  | Cons (hd1, tl_begetter1), Cons (hd2, tl_begetter2) ->
      let part1 = product_wrong (tl_begetter1 ()) u2 in
      let part2 = map (fun y -> (hd1, y)) (tl_begetter2 ()) in
      Cons ((hd1, hd2), fun () -> intertwin part1 part2)
```

1. Montrer que `product_wrong u1 u2` ne termine pas toujours, et qu'il suffit de déplacer quelques mots pour le corriger.
2. Signification du `0` dans `return 0` (C) ou `exit 0` (OCaml).
3. Énoncer le théorème de Turing sur l'arrêt.
4. On admet une fonction `run : string -> int -> int` qui termine toujours et renvoie 0 si le code x termine sans erreur en moins de n étapes, 1 sinon. Construire ou réfuter l'existence de `filter (f:'a -> bool) (u:'a stream) : 'a stream` (éléments de `u` vérifiant `f`, dans l'ordre ; par exemple `filter is_negative ones` vaut `Nil`).

Décoder l'énoncé

Ce que l'énoncé te donne.

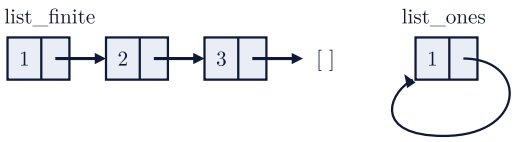
- OCaml est strict : les arguments d'une fonction sont évalués avant l'appel.
- `fun () -> e` retarde le calcul de `e`.

Ce qu'on te demande. Comprendre pourquoi certaines définitions récursives bouclent, et programmer avec des flots paresseux.

Corrigé

Question 1

Étape 1. Q1. Les listes en mémoire



chaque maillon : (tête, pointeur vers la queue) ; stockés dans le tas

`list_finite` : trois maillons terminés par la liste vide. `list_ones` : un seul maillon dont la queue pointe sur lui-même (liste cyclique).

`list_finite` est une chaîne de trois maillons, le dernier pointant vers la liste vide. `list_ones` est un **seul** maillon dont la queue pointe sur lui-même : une liste cyclique, donc « infinie » quand on la parcourt.

Ces blocs sont stockés dans le **tas** (zone de mémoire gérée par le ramasse-miettes d'OCaml), pas dans la pile.

Question 2

Étape 1. Q2. Stack overflow

Stack se traduit par **pile** (la pile d'appels).

Déroulement : pour évaluer `list_fibo_with_internal_state 0 1`, il faut construire `0 :: (...)`, donc d'abord évaluer la queue `list_fibo_with_internal_state 1 1` (OCaml est strict : on évalue les arguments d'un constructeur avant de le construire). Celle-ci demande `list_fibo_with_internal_state 1 2`, puis `2 3`, `3 5`… Aucun appel ne se termine, et chacun empile un bloc d'activation : la pile déborde.

Pourquoi `list_ones` marche : `let rec list_ones = 1 :: list_ones` est une définition récursive de **valeur** (un constructeur appliqué au nom défini), qu'OCaml traite en créant directement le maillon cyclique, sans appel de fonction.

Question 3

Étape 1. Q3. Compilation ou exécution ?

La **compilation réussit** : le code est syntaxiquement correct et bien typé (`list_fibo : int list`). L'erreur se produit à l'**exécution** de `./fibo` : la récursion infinie épuise la pile et le programme s'arrête avec l'exception `Stack_overflow` (ou une erreur de segmentation, selon le système).

Question 4

Étape 1. Q4. Le flot des entiers

```
let integers : int stream =
  let rec from n = Cons (n, fun () -> from (n + 1)) in
  from 0
```

L'appel `from (n + 1)` est retardé par `fun () ->` : la définition termine immédiatement.

Question 5

Étape 1. Q5. range

```
let rec range (a:int) (b:int) : int stream =
  if a >= b then Nil
  else Cons (a, fun () -> range (a + 1) b)
```

Question 6

Étape 1. Q6. Toute suite est-elle un flot ?

Non, pas forcément. Un flot est défini par un programme OCaml, c'est-à-dire un texte fini : il y a un ensemble **dénombrable** de tels programmes. L'ensemble des suites de chaînes n'est pas dénombrable : déjà les suites à valeurs dans $\{\texttt{"0"}, \texttt{"1"}\}$ sont en bijection avec $\{0,1\}^{\mathbb{N}}$, non dénombrable (argument diagonal de Cantor). Il existe donc des suites qu'aucun flot ne code.

Quand c'est possible : si la suite est **calculable**, c'est-à-dire s'il existe une fonction `sigma : int -> string` qui calcule σ_n, alors

```
let stream_of_sequence (sigma : int -> string) : string stream =
  let rec from n = Cons (sigma n, fun () -> from (n + 1)) in
  from 0
```

Question 7

Étape 1. Q7. hd

```
let hd (u : 'a stream) : 'a =
  match u with
  | Nil -> failwith "hd"
  | Cons (x, _) -> x
```

Question 8

Étape 1. Q8. tl

```
let tl (u : 'a stream) : 'a stream =
  match u with
  | Nil -> failwith "tl"
  | Cons (_, f) -> f ()
```

C'est l'appel `f ()` qui calcule effectivement la suite du flot.

Question 9

Étape 1. Q9. of_list

```
let rec of_list (l : 'a list) : 'a stream =
  match l with
  | [] -> Nil
  | x :: q -> Cons (x, fun () -> of_list q)
```

Question 10

Étape 1. Q10. iter

```
let rec iter (f : 'a -> unit) (t : int) (u : 'a stream) : unit =
  if t > 0 then
    match u with
    | Nil -> ()                          (* moins de t valeurs existantes *)
    | Cons (x, g) -> f x; iter f (t - 1) (g ())
```

On ne force la queue `g ()` que si l'on en a besoin. Si $t \leqslant 0$, rien ne se passe. La récursion est terminale : pas de risque de débordement de pile.

Question 11

Étape 1. Q11. map

```
let rec map (f : 'a -> 'b) (u : 'a stream) : 'b stream =
  match u with
  | Nil -> Nil
  | Cons (x, g) -> Cons (f x, fun () -> map f (g ()))
```

Question 12

Étape 1. Q12. zip

```
let is_nil u = match u with Nil -> true | Cons _ -> false

let rec zip (w : 'a stream array) : 'a array stream =
  if Array.exists is_nil w then Nil
  else Cons (Array.map hd w, fun () -> zip (Array.map tl w))
```

On teste le vide par filtrage (`is_nil`) plutôt que par `u = Nil` : l'égalité structurelle pourrait tenter de comparer des fonctions. Les queues ne sont calculées (`Array.map tl w`) que lorsque l'on demande la suite.

Question 13

Étape 1. Q13. intertwin

```
let rec intertwin (u : 'a stream) (v : 'a stream) : 'a stream =
  match u with
  | Nil -> v                                    (* u épuisé : on continue avec v *)
  | Cons (x, g) -> Cons (x, fun () -> intertwin v (g ()))   (* on alterne *)
```

Vérification sur `intertwin (range 0 2) fibo` : 0, puis F_0 (rôles échangés), puis 1, puis F_1 ; `range` est alors épuisé et l'on continue avec $F_2, F_3, \ldots$: c'est l'exemple de l'énoncé.

Question 14

Étape 1. Q14. Corriger product_wrong

Non-terminaison : `part1 = product_wrong (tl_begetter1 ()) u2` est calculé **avant** de renvoyer le `Cons`. Cet appel calcule à son tour le `part1` suivant, et ainsi de suite le long de tout `u1`. Si `u1` est infini (par exemple `ones`) et `u2` non vide, la récursion ne s'arrête jamais.

Correction : il suffit de déplacer `fun () ->` pour que `part1` et `part2` ne soient calculés qu'à la demande :

```
let rec product u1 u2 =
  match u1, u2 with
  | _, Nil | Nil, _ -> Nil
  | Cons (hd1, tl_begetter1), Cons (hd2, tl_begetter2) ->
      Cons ((hd1, hd2), fun () ->
        let part1 = product (tl_begetter1 ()) u2 in
        let part2 = map (fun y -> (hd1, y)) (tl_begetter2 ()) in
        intertwin part1 part2)
```

Chaque appel renvoie maintenant un `Cons` immédiatement. Chaque couple (u_i, v_j) apparaît : les couples avec u_0 en tête sont dans `part2` ou en tête, les autres dans `part1`, et `intertwin` alterne entre les deux, donc tout élément d'un des deux flots finit par sortir.

Question 15

Étape 1. Q15. Le 0 de return 0

C'est le **code de retour** du processus, transmis au système d'exploitation (ou au processus parent, par exemple le terminal). Par convention, 0 signifie que le programme s'est terminé **sans erreur** ; une valeur non nulle signale un échec.

Question 16

Étape 1. Q16. Théorème de Turing

> Il n'existe aucun programme qui, recevant le code d'un programme P et une entrée x, termine toujours et répond correctement à la question « l'exécution de P sur x termine-t-elle ? ». Le problème de l'arrêt est **indécidable**.

Question 17

Étape 1. Q17. Un filtre ne peut pas exister

On réfute l'existence d'une telle fonction `filter` (qui termine toujours, puisque `filter is_negative ones` doit valoir `Nil`).

Supposons qu'elle existe. Pour un code source x, on définit le flot

```
let witness (x : string) : int stream = map (run x) integers
(* terme n : 0 si x termine en moins de n étapes, 1 sinon *)

let halts (x : string) : bool =
  match filter (fun v -> v = 0) (witness x) with
  | Nil -> false        (* aucun 0 : x ne termine jamais *)
  | Cons _ -> true      (* un 0 existe : x termine *)
```

`run` termine toujours, donc `witness x` est un flot infini bien défini. Le flot filtré est vide si et seulement si aucun terme ne vaut 0, c'est-à-dire si x ne termine pas. Comme `filter` termine, `halts` termine toujours et décide l'arrêt, ce qui contredit le théorème de Turing.

Remarque : on peut écrire un filtre **partiel**, qui cherche le prochain élément valide et boucle s'il n'y en a plus (`filter is_negative ones` ne terminerait alors pas) ; c'est la spécification « vaut `Nil` » qui est impossible.

Le piège

En Q14, croire que `map` ou `intertwin` est en cause : c'est l'appel récursif `product_wrong` calculé avant le `Cons` qui ne termine pas.

La méthode

Flot paresseux : renvoyer tout de suite un `Cons` dont la queue est une fonction ; ne jamais forcer une queue avant d'en avoir besoin.

Rappels de cours utilisés

- **Flots paresseux.** `Cons (x, f)` : la queue n'est calculée que lorsqu'on appelle `f ()`. Le corps d'une fonction `fun () -> ...` n'est pas évalué à sa création : c'est ce qui permet des suites infinies sans boucle infinie.
- **Indécidabilité de l'arrêt.** Théorème de Turing : il n'existe pas de programme qui, pour tout programme P et toute entrée x, termine et répond correctement à « P termine-t-il sur x ? ».